

REPORT

제목: 스트림릿(**Streamlit**) 활용 가이드 레포트

과목명: 인공지능시스템설계

담당 교수: 이혜정 교수님

학과: 인공지능 소프트웨어학과

학번: **2603340023**

이름: 김광민

제출일: **2026. 04. 29**

목차

1.스트림릿(**Streamlit**) 개요 및 환경 구축

2.스트림릿 시작 및 기본 코드 구조

3.스트림릿 기본 **API** 활용

- 3.1 텍스트 및 데이터 출력 (Write, Magic Command)
- 3.2 입력 위젯 (Input Widgets)
- 3.3 폼(Form) 및 사이드바(Sidebar) 구성

4.스트림릿의 데이터 흐름(**Data Flow**) 및 캐싱

5.외부 시각화 라이브러리 연동

1. 스트림릿(Streamlit) 개요 및 환경 구축

스트림릿은 상호작용이 가능한 웹 대시보드를 생성해 주는 웹 대시보드 개발 라이브러리입니다.

개발환경

- Conda 가상환경 : wordfreq
- 데이터 분석: numpy, pandas, matplotlib
- 한글 처리 : konlpy
- 워드클라우드 : wordcloud
- 웹대시보드 : streamlit

IDE : Visual Studio Code

웹 대시보드 개발 라이브러리

- 웹 대시보드를 생성하여 상호작용을 가능하게 만들어주는 라이브러리
- 공식 문서: <https://docs.streamlit.io/>

개발 환경을 구축하려면 파이썬 환경에서 `pip install streamlit` 명령어를 통해 설치할 수 있습니다.

설치가 정상적으로 완료되었는지 확인하려면 터미널에 `streamlit hello`를 입력하여 테스트 앱을 실행해 볼 수 있습니다.

2. 스트림릿 시작 및 기본 코드 구조

실행 방법

작성한 파이썬 파일을 스트림릿으로 실행하는 기본 명령어는 “`streamlit run 파일명.py`”입니다.

코드 상단에 `import streamlit as st`를 추가하여 라이브러리를 불러옵니다.

스트림릿 코드는 일반 파이썬 코드와 특별한 구분자나 지시자 없이 혼합하여 자연스럽게 사용할 수 있습니다.

3. 스트림릿 기본 API 활용

3.1 텍스트 및 데이터 출력 (Write, Magic Command)

- `st.write()` 함수는 입력된 데이터의 타입(Data Type)에 맞게 자동으로 시각화하여 화면에 출력합니다.
- 매직 커맨드(Magic command) 기능을 지원하여, 특별한 출력 명령어나 `st.write()`를 명시하지 않아도 데이터 타입에 맞추어 자동으로 화면에 출력됩니다.

TEXT 출력

구분	함수명	설명 및 용도	시각적 특징
텍스트 출력	<code>st.title()</code>	페이지의 메인 제목	가장 큰 글자 크기
	<code>st.header()</code>	섹션별 주제	<code>title</code> 보다 작은 크기

	<code>st.subheader()</code>	하위 섹션 제목	일반 글자보다 약간 큼
	<code>st.text()</code>	일반 본문 텍스트	고정 폭 글꼴 등 일반 텍스트
상태 알림	<code>st.success()</code>	작업이 성공적으로 완료됨을 알림	초록색(Green) 배경
	<code>st.warning()</code>	주의나 경고 사항을 전달	노란색(Yellow) 배경
	<code>st.info()</code>	일반적인 정보나 안내 제공	파란색(Blue) 배경
	<code>st.error()</code>	오류 발생을 알림	빨간색(Red) 배경

3.2 입력 위젯 (Input Widgets)

- 사용자와 상호작용하기 위한 다양한 위젯을 제공합니다.
- 클릭 여부를 반환하는 `button()`, 체크 여부를 반환하는 `checkbox()`를 사용할 수 있습니다.

- 항목 선택을 위해 `radio()`, `selectbox()`, 복수 선택이 가능한 `multiselectbox()`가 제공되며, 선택한 항목의 텍스트나 리스트를 반환합니다.
- 이 외에도 수치 범위를 조절하는 `slider()`, 문자열을 입력받는 `text_input()`, 숫자를 입력받는 `number_input()` 등을 활용할 수 있습니다.

함수명	주요 인자(Arguments)	반환 값(Return Value)
<code>button()</code>	<code>'button_name'</code>	클릭 여부 (True/False)
<code>radio()</code>	<code>'message'</code> , <code>list_of_options</code>	선택한 항목의 text
<code>checkbox()</code>	<code>'message'</code>	체크 여부 (True/False)
<code>selectbox()</code>	<code>'message'</code> , <code>list_of_options</code>	선택한 항목의 text
<code>multiselect()</code>	<code>'message'</code> , <code>list_of_options</code>	선택한 항목들의 list
<code>slider()</code>	<code>'message'</code> , <code>start</code> , <code>end</code> , <code>default</code> , <code>step</code>	조절된 숫자 값

<code>text_input()</code>	<code>'message'</code>	입력한 문자열
<code>number_input()</code>	<code>'message'</code>	입력한 숫자

3.3 폼(Form) 및 사이드바(Sidebar) 구성

- 여러 위젯의 입력값을 한 번에 묶어서 처리하고 싶을 때는 폼(Form) 기능을 사용합니다.
- `st.form(key)`로 폼을 정의하고, 내부에 포함될 위젯들은 `with` 블록 안에 작성합니다.
- 폼 내부에는 반드시 `st.form_submit_button(label)`을 통해 제출 버튼을 정의해야 하며, 이 버튼이 눌렸을 때(`if submitted:`) 입력값들을 한꺼번에 처리합니다.
- 화면 왼쪽에 위젯이나 정보를 배치하고 싶다면 `st.sidebar.` 속성을 붙여 기존과 동일한 메서드를 사용하여 사이드바를 구성할 수 있습니다.

4. 스트림릿의 데이터 흐름(Data Flow) 및 캐싱

- 스트림릿은 화면이 업데이트되어야 하는 상황(소스 코드가 수정되거나, 사용자가 위젯을 조작하여 상호작용이 발생한 경우)이 발생하면 파이썬 코드 전체를 처음부터 다시 실행하는 방식으로 동작합니다.
- 매번 불필요하게 코드가 재실행되는 것을 방지하고 성능을 높이기 위해 데코레이터인 `@st.cache_data`를 사용할 수 있습니다.

5. 외부 시각화 라이브러리 연동

- `Matplotlib`이나 `Wordcloud` 등 다른 데이터 시각화 라이브러리와 쉽게 연동할 수 있습니다.

- 예를 들어 Matplotlib의 경우 `fig, ax = plt.subplots()`로 생성한 그래프 객체를 `st.pyplot(fig)` 함수에 전달하여 스트림릿 웹 화면에 출력할 수 있습니다.